

Project Requirements Document (PRD)

IT Awareness Club Website

Project Name	IT Awareness Club Website
Document Version	1.0
Date	July 2026
Platform	Responsive Web Application
Stack	HTML5, CSS3, Vanilla JavaScript (no backend, no database)

1. Project Overview

The IT Awareness Club Website is a fully static, responsive web application that serves as the club's digital home. It introduces the club to prospective and current students, showcases activities and achievements, and preserves the club's legacy by documenting each generation of members.

Because the site is frontend-only, all content is stored in the codebase itself — either directly in HTML or in local JSON files loaded by JavaScript. This keeps hosting free (GitHub Pages, Netlify, Vercel), eliminates security and maintenance overhead of a server, and is realistic for a student organization to maintain over time.

Design rationale: A static architecture was chosen deliberately. Student clubs have high member turnover; a plain HTML/CSS/JS site can be maintained by any member with basic web skills, whereas a backend or CMS would create a knowledge bottleneck. Storing dynamic-feeling content (activities, achievements, members) in JSON files gives editors a single, simple place to update content without touching layout code.

2. Objectives

- Introduce the club** — communicate the club's identity, mission, and vision within 5 seconds of landing on the homepage.
- Attract new members** — provide clear calls-to-action (CTA) that guide interested students toward joining or contacting the club.
- Showcase activities** — present workshops, events, and projects in a visual, browsable format.
- Celebrate achievements** — document awards, certificates, competitions, and milestones in a chronological timeline.
- Preserve club history** — record every generation of members so the club's legacy is visible and alumni feel recognized.
- Demonstrate technical skill** — the site itself should be a portfolio piece proving the club's competence in modern frontend development.

3. Target Users

User Type	Description	Primary Goals
Prospective members	Students curious about the club	Understand what the club does; find out how to join
Current members	Active club participants	Check upcoming activities and announcements
Alumni / past generations	Former members	Revisit their generation's page; see the club's growth
Faculty & sponsors	Advisors, judges, potential partners	Evaluate the club's credibility via achievements

Primary persona: A first-year student browsing on a mobile phone between classes. This drives two hard requirements: mobile-first design and fast load times on average connections.

4. Functional Requirements

FR-1: Global (all pages)

- FR-1.1 — Sticky navigation bar with logo, links to all four pages, and active-page indicator.
- FR-1.2 — Hamburger menu on screens < 768px with an animated open/close toggle.
- FR-1.3 — Footer with contact info, social media links, quick links, and copyright.
- FR-1.4 — Back-to-top button appears after scrolling ~400px; smooth-scrolls to top.
- FR-1.5 — Loading animation (lightweight CSS spinner or logo animation) shown until page content is ready.
- FR-1.6 — Scroll-reveal animations on section entry using IntersectionObserver.

FR-2: Home page

- FR-2.1 — Hero section with club name, tagline, background image/gradients, and two CTA buttons ("Explore Activities", "Join Us").
- FR-2.2 — Club introduction section (short "who we are" paragraph + image).
- FR-2.3 — Mission and vision presented as two side-by-side cards.
- FR-2.4 — Featured activities: 3 highlighted activity cards linking to the Activities page.

- FR-2.5 — Latest news/announcements: 2-3 recent items rendered from `news.json`.
- FR-2.6 — Statistics strip (members count, events held, awards won) with count-up animation on scroll into view.

FR-3: Club Activities page

- FR-3.1 — Grid of activity cards, each with image, title, date, category badge, and short description.
- FR-3.2 — Filter buttons by category (e.g., All / Workshops / Competitions / Community) — client-side filtering of the rendered cards.
- FR-3.3 — Live search input filtering cards by title/description as the user types.
- FR-3.4 — Gallery section: responsive image grid with a lightbox (click to enlarge, keyboard navigable, close on Esc/overlay click).
- FR-3.5 — Activities rendered from `activities.json` so members can add events without editing HTML.
- FR-3.6 — "No results" empty state when a search/filter matches nothing.

FR-4: What We Achieved page

- FR-4.1 — Category sections: Awards, Certificates, Competitions, Events, Milestones.
- FR-4.2 — Vertical timeline of achievements, newest first, with alternating left/right entries on desktop and single-column on mobile.
- FR-4.3 — Each timeline entry: year marker, title, description, optional image/certificate thumbnail.
- FR-4.4 — Certificate thumbnails open in the same lightbox component used in the gallery.
- FR-4.5 — Data driven from `achievements.json`.

FR-5: About page

- FR-5.1 — Club history section including founding year and origin story.
- FR-5.2 — Club objectives as an icon + text list.
- FR-5.3 — Organization structure: simple hierarchy diagram (President → Vice President → Team Leads → Members) built with CSS grid/flexbox — no images required, so it stays editable.
- FR-5.4 — Generations of members (see below).
- FR-5.5 — Data driven from `members.json`.

FR-6: Generations presentation (recommended approach)

Recommendation: Tabbed generation selector + member cards, with an accordion fallback on mobile.

- A horizontal, scrollable tab bar of years ("2023 · 2024 · 2025 · 2026"), most recent selected by default.
- Selecting a year reveals a grid of member cards (photo/avatar, name, role) with a subtle fade-in.
- On mobile, tabs remain but the card grid collapses to 2 columns.

Why this over pure accordions or a timeline:

- *Tabs* scale gracefully as generations accumulate (a timeline gets very long after 5+ years; stacked accordions bury older generations).
- *Cards* give each member visual recognition — important for a page whose emotional purpose is honoring people.
- Default-selecting the newest generation serves the most common visitor intent while keeping history one tap away.
- Purely optional enhancement: a small "legacy strip" above the tabs showing the founding year → present as a mini-timeline, giving historical context without the maintenance cost of a full timeline.

5. Non-Functional Requirements

Category	Requirement
Performance	First Contentful Paint < 1.5s on 4G; total page weight < 1.5 MB; Lighthouse Performance ≥ 90
Responsiveness	Fully usable from 320px to 1920px+ viewports
Accessibility	WCAG 2.1 AA: contrast ≥ 4.5:1, keyboard navigable, semantic HTML, alt text on all images
Browser support	Latest 2 versions of Chrome, Firefox, Edge, Safari (desktop + mobile)
Maintainability	Content in JSON files; CSS custom properties for theming; commented, modular JS
SEO	Unique title/meta description per page; semantic headings; Open Graph tags
Security	No user data collected; external links use <code>rel="noopener noreferrer"</code>
Hosting	Deployable to any static host (GitHub Pages recommended — free for student orgs)

6. Website Sitemap

```
IT Awareness Club Website
|
|— index.html ..... Home
|   |— Hero
|   |— Introduction
|   |— Mission & Vision
|   |— Featured Activities
|   |— Latest News
|   |— Stats / CTA
|   |— Footer
|
|— activities.html ..... Club Activities
|   |— Page header
|   |— Search + Filter bar
|   |— Activity cards grid
```

	└─ Gallery (+ lightbox)
	└─ Footer
└─ achievements.html	 What We Achieved
	└─ Page header
	└─ Achievement categories
	└─ Timeline
	└─ Footer
└─ about.html	 About
	└─ Page header
	└─ Club history
	└─ Objectives
	└─ Organization structure
	└─ Generations (tabs + member cards)
	└─ Join CTA
	└─ Footer

7. Navigation Flow

Primary flow (prospective member): Home (hero) → "Explore Activities" CTA → Activities page → back to Home → "Join Us" CTA → contact info in footer / join section.

Secondary flows:

- Home → Featured activity card → Activities page (pre-scrolled or filtered to that item).
- Home → News item → relevant page (Activities or Achievements).
- Any page → sticky navbar → any other page (max 1 click between any two pages — flat architecture, intentional for a 4-page site).
- Long page → Back-to-top button → page top.

Navigation rules:

- Current page highlighted in navbar (underline or color accent).
- Mobile menu closes automatically after a link is tapped.
- Logo always links to Home.
- Footer repeats primary navigation (users who reach the bottom shouldn't have to scroll back up to move on).

8. Detailed Page Requirements

8.1 Home (index.html)

Section	Content	Behavior
Hero	H1 club name, tagline, 2 CTA buttons, background gradient/image with dark overlay	Text fades in on load; buttons have hover lift; scroll-down indicator
Introduction	~80-word "who we are" + supporting image	Scroll-reveal: image slides from left, text from right
Mission & Vision	Two cards, each with icon, heading, 2-3 sentences	Hover: soft shadow deepens, slight lift
Featured Activities	3 cards pulled from the top of activities.json	Card click → activities.html
Latest News	2-3 announcement items with date badges	Rendered from news.json
Stats strip	Members, events, awards counters	Count-up animation when scrolled into view (runs once)
Join CTA	Full-width band with button	Links to contact section / social media
Footer	Contact, socials, quick links, © year (auto via JS)	Social icons have hover color transition

8.2 Club Activities (activities.html)

Section	Content	Behavior
Page header	Title + one-line description	Simple fade-in
Search + filters	Text input, category buttons	Debounced (250ms) live search; filter buttons toggle active state; search and filter combine (AND logic)
Activity grid	Cards: image (16:9), category badge, title, date, ~25-word description	3 columns → 2 (tablet) → 1 (mobile); hover: image zooms slightly within its frame
Empty state	Illustration/icon + "No activities match your search"	Shown when 0 results; includes a "Clear filters" button
Gallery	Masonry-style or uniform grid of event photos	Click opens lightbox; lightbox supports ←/→ keys, Esc to close, focus trapped while open

8.3 What We Achieved (achievements.html)

Section	Content	Behavior
Page header	Title + summary line ("X awards · Y competitions · since 20XX")	Counts computed from JSON
Category chips	Awards / Certificates / Competitions / Events / Milestones	Clicking a chip filters the timeline
Timeline	Vertical line, alternating entries (desktop), single column (mobile); each entry: year node, icon by category, title, description, optional thumbnail	Entries reveal as they scroll into view; thumbnails open in lightbox

Design rationale: One unified timeline with category filters beats five separate lists — it tells a growth story ("look how far we've come") and avoids five short, sparse sections in the club's early years.

8.4 About (about.html)

Section	Content	Behavior
History	Founding year, origin story, growth summary	Scroll reveal
Objectives	4–6 items, each icon + short text	Staggered reveal (80ms delay between items)
Organization structure	CSS-built hierarchy: President → VP → Team Leads → Members	Pure HTML/CSS so it's editable; collapses to a stacked list on mobile
Generations	Year tabs + member card grid (photo, name, role)	Newest year default; switching tabs cross-fades the grid; cards use placeholder avatar if no photo
Join CTA	Reused CTA band component	Same as Home

9. UI Component List

Layout components

- 1. Navbar (sticky, with mobile hamburger)
- 2. Footer
- 3. Page header / breadcrumb banner
- 4. Section wrapper (consistent max-width + padding)

Content components 5. Button (primary, secondary/outline, icon variants) 6. Activity card 7. Member card 8. News item 9. Mission/Vision card 10. Timeline entry 11. Stat counter 12. Category badge / filter chip 13. Objective list item (icon + text) 14. Org-chart node

Interactive components 15. Search input (with clear button) 16. Tab bar (generations) 17. Lightbox / image modal 18. Back-to-top button 19. Loading spinner / preloader 20. Empty-state block

Component rule: every component is a CSS class family (e.g., `.card`, `.card_image`, `.card--featured`) following BEM-style naming, defined once in `components.css` and reused across pages. This is the single biggest maintainability lever in a no-framework project.

10. Recommended Color Palette

A tech-forward blue base (trust, technology) with a cyan accent (energy, modernity) — appropriate for an IT club without feeling corporate.

Role	Color	Hex	Usage
Primary	Deep Blue	#1E3A8A	Navbar, headings, primary buttons
Primary Dark	Navy	#172554	Footer, hero overlay
Accent	Cyan	#06B6D4	CTAs, links, hover states, badges
Accent Light	Sky	#67E8F9	Highlights, gradient stops
Background	Off-white	#F8FAFC	Page background
Surface	White	#FFFFFF	Cards, navbar
Text Primary	Slate 900	#0F172A	Body headings/text
Text Secondary	Slate 500	#64748B	Dates, captions, muted text
Success	Emerald	#10B981	Achievement badges
Warning/Star	Amber	#F59E0B	Award icons

Implement as CSS custom properties:

```
:root {
  --color-primary: #1E3A8A;
  --color-primary-dark: #172554;
  --color-accent: #06B6D4;
  --color-bg: #F8FAFC;
  --color-surface: #FFFFFF;
  --color-text: #0F172A;
  --color-text-muted: #64748B;
  --radius-md: 12px;
  --radius-lg: 20px;
  --shadow-soft: 0 4px 16px rgba(15, 23, 42, 0.08);
  --shadow-hover: 0 12px 28px rgba(15, 23, 42, 0.14);
  --transition: 0.25s ease;
}
```

Contrast check: #1E3A8A on white = 10.3:1, #64748B on white = 4.76:1, white on #06B6D4 fails for small text — so cyan buttons must use dark text (#0F172A , 8.7:1) or be reserved for large text/icons.

11. Typography Recommendations

Role	Font	Weights	Rationale
Headings	Poppins	600, 700	Geometric, modern, friendly — suits a student tech club
Body	Inter	400, 500	Exceptional screen legibility at small sizes
Optional code/accents	JetBrains Mono	400	Tech identity touches (e.g., hero tagline styled like code)

Both served via Google Fonts with `display=swap` , limited to the listed weights (each extra weight $\approx$ 15–30 KB).

Type scale (fluid, using `clamp()`):

```
h1 { font-size: clamp(2rem, 5vw, 3.5rem); } /* Hero */
h2 { font-size: clamp(1.5rem, 3.5vw, 2.25rem); } /* Sections */
h3 { font-size: clamp(1.15rem, 2.5vw, 1.5rem); } /* Cards */
body { font-size: 1rem; line-height: 1.65; }
small { font-size: 0.875rem; }
```

Line length capped at $\sim$ 70 characters (`max-width: 65ch`) for paragraphs.

12. Icon Recommendations

Primary choice: [Lucide](#) — free, MIT-licensed, consistent 24px stroke icons, available as plain SVG (no framework needed). Alternative: Font Awesome Free 6 if the club prefers a single `<script>` include.

Recommended usage: inline SVG (copy-paste from Lucide) rather than icon fonts — better performance, no extra request, colorable via `currentColor` .

Context	Icons
Navigation	menu , x , chevron-up (back-to-top)
Mission/Vision	target , eye
Activities	calendar , search , filter , image
Achievements	trophy , award , medal , flag , star
About	history / clock , users , list-checks , network
Footer/social	facebook , instagram , youtube , github , mail , map-pin , send (Telegram)

13. Responsive Design Guidelines

Approach: mobile-first CSS (base styles for mobile, `min-width` media queries scale up).

Breakpoint	Target	Key changes
Base (320–639px)	Phones	1-column grids, hamburger menu, stacked hero, timeline single-column
<code>min-width: 640px</code>	Large phones	2-column card grids
<code>min-width: 768px</code>	Tablets	Full navbar appears, 2-col mission/vision
<code>min-width: 1024px</code>	Laptops	3-column card grids, alternating timeline
<code>min-width: 1280px</code>	Desktops	Max content width 1200px, larger spacing scale

Rules:

- Use CSS Grid with `repeat(auto-fit, minmax(280px, 1fr))` for card grids — grids adapt with almost no media queries.
- Tap targets $\geq$ 44×44px on mobile.
- Images: `max-width: 100%` , explicit `width / height` attributes to prevent layout shift, `srcset` for hero images.
- Test at 320px (small phones), 768px (tablet portrait), and with browser zoom at 200%.
- Never hide core content on mobile — reflow it.

14. Folder Structure

```
it-awareness-club/
├── index.html
├── activities.html
├── achievements.html
├── about.html
├──
├── css/
│   ├── variables.css      # Custom properties (colors, spacing, shadows)
│   ├── base.css           # Reset, typography, global element styles
│   ├── components.css     # Buttons, cards, badges, navbar, footer, lightbox
│   ├── pages.css          # Page-specific sections (hero, timeline, generations)
│   └── animations.css     # Keyframes, reveal classes, transitions
├── js/
│   ├── main.js            # Navbar, back-to-top, preloader, footer year (all pages)
│   ├── reveal.js          # IntersectionObserver scroll-reveal utility
│   ├── activities.js      # Render cards from JSON, search, filter
│   ├── achievements.js    # Render + filter timeline
│   ├── about.js           # Generations tabs rendering
│   └── lightbox.js        # Shared lightbox component
├── data/
│   ├── activities.json
│   ├── achievements.json
│   ├── members.json      # { "2025": [ { name, role, photo } ], ... }
│   └── news.json
├──
└── assets/
```

```
|
| |
| | | images/
| | | | hero/
| | | | activities/
| | | | gallery/
| | | | achievements/
| | | | members/
| | |
| | icons/ # Inline-able SVGs
| | logo/
|
| favicon.ico
| README.md # How to add an activity/member (for future maintainers)
```

Rationale: CSS split by concern (not by page) maximizes reuse; JS split by page keeps files small and lets each HTML page load only what it needs. The `README.md` with "how to update content" instructions is arguably the most important maintainability deliverable for a club with yearly turnover.

15. Suggested Animations & Interactions

Animation	Where	Implementation	Notes
Preloader	Page load	CSS spinner, removed on <code>window.load</code>	Cap with a 2.5s timeout fallback so it never traps users
Hero entrance	Home	CSS keyframe fade-up, staggered via <code>animation-delay</code>	Runs once
Scroll reveal	All sections	<code>IntersectionObserver</code> adds <code>.revealed</code> class → CSS transition	Threshold 0.15; unobserve after reveal
Staggered items	Objectives, cards	<code>transition-delay: calc(var(--i) * 80ms)</code>	Set <code>--i</code> inline per item
Button hover	All buttons	<code>translateY(-2px)</code> + shadow deepen	0.25s ease
Card hover	Activity/member cards	Lift + inner image <code>scale(1.05)</code> with <code>overflow: hidden</code>	GPU-friendly transforms only
Navbar shrink	On scroll	Toggle <code>.scrolled</code> class → reduced padding + shadow	Use a passive scroll listener
Count-up stats	Home stats	<code>requestAnimationFrame</code> counter triggered by observer	Runs once
Smooth scrolling	Anchor links	<code>html { scroll-behavior: smooth; }</code>	Zero JS
Timeline draw	Achievements	Line/entries fade-slide in as they enter viewport	
Lightbox	Gallery, certificates	Fade + slight scale on open	Trap focus, restore on close
Back-to-top	All pages	Fade in after 400px scroll	

Two hard rules:

1. Animate only `transform` and `opacity` (compositor-friendly; never animate `top/left/width/height`).
2. Respect user preferences:

```
@media (prefers-reduced-motion: reduce) {
  *, *::before, *::after {
    animation-duration: 0.01ms !important;
    transition-duration: 0.01ms !important;
    scroll-behavior: auto !important;
  }
}
```

16. SEO Recommendations

1. **Unique metadata per page** — e.g., Home: `<title>IT Awareness Club – Learn, Build, Share</title>` with a ~155-character meta description; repeat pattern for all four pages.
2. **Semantic structure** — one `<h1>` per page; logical `<h2>/<h3>` hierarchy; use `<header>`, `<nav>`, `<main>`, `<section>`, `<article>`, `<footer>`.
3. **Open Graph + Twitter Card tags** — so shared links on Facebook/Telegram/Discord show the club logo and description (high-impact for a student audience that shares everything via chat apps).
4. **Descriptive URLs & filenames** — `activities.html`, images like `web-workshop-2025.jpg`, not `IMG_2041.jpg`.
5. **Alt text on every image** — doubles as accessibility.
6. **Structured data (JSON-LD)** — `Organization` schema on Home with name, logo, and social profiles.
7. `sitemap.xml` + `robots.txt` — trivial for 4 pages; helps indexing on GitHub Pages.
8. **Canonical URLs** and a proper 404 page.
9. **Performance is SEO** — Core Web Vitals feed into ranking; see Section 18.

17. Accessibility Recommendations

1. **Semantic HTML first** — real `<button>` for buttons/filters (not clickable `<div>` s), `<a>` for navigation.
2. **Keyboard support** — every interactive element reachable and operable by Tab/Enter/Space; visible `:focus-visible` outline (2px accent); logical tab order.
3. **Skip link** — "Skip to main content" as the first focusable element.
4. **Lightbox a11y** — focus trapped while open, `Esc` closes, focus returns to the triggering thumbnail, `role="dialog"` + `aria-modal="true"`.
5. **Mobile menu a11y** — hamburger is a `<button>` with `aria-expanded` and `aria-controls`.
6. **Contrast** — all text $\geq 4.5:1$ (see palette notes: dark text on cyan buttons).
7. **Alt text** — meaningful for content images; `alt=""` for decorative ones.
8. **Forms/inputs** — search input has an associated `<label>` (visually hidden is fine).
9. **Dynamic updates announced** — filter results count in an `aria-live="polite"` region ("6 activities shown").

- 10. **Reduced motion** honored (Section 15).
- 11. **Don't rely on color alone** — active filter chips also change shape/weight/icon.
- 12. **Test** with keyboard-only navigation, a screen reader (NVDA/VoiceOver), and Lighthouse's accessibility audit (target ≥ 95).

18. Performance Optimization Tips

- 1. **Images are 90% of the battle** — convert to WebP (JPEG fallback optional), resize to actual display dimensions (cards need ~600px wide, not 4000px camera originals), target < 120 KB per card image.
- 2. **Lazy-load** everything below the fold: `loading="lazy"` on `<img>`; hero image loads eagerly with `fetchpriority="high"`.
- 3. **Prevent layout shift** — set `width / height` (or `aspect-ratio`) on all images.
- 4. **Minimize requests** — inline critical SVG icons; 4-5 CSS files can be concatenated for production if needed (optional at this scale).
- 5. **Fonts** — only needed weights, `display=swap`, preconnect to `fonts.gstatic.com`.
- 6. **JS discipline** — `defer` all scripts; passive scroll listeners; debounce search input; no libraries (the entire feature set above is achievable in < 15 KB of vanilla JS).
- 7. **Cache-friendly hosting** — GitHub Pages/Netlify serve with gzip/HTTP2 automatically.
- 8. **Budget & audit** — total page < 1.5 MB, JS < 50 KB, run Lighthouse before every release; treat a Performance score < 90 as a bug.

19. Development Roadmap

Estimated for a small student team working part-time (~6 weeks total). Adjust to team availability.

Milestone	Duration	Deliverables	Definition of done
M1 — Foundation & design system	Week 1	Folder structure, <code>variables.css</code> , <code>base.css</code> , navbar + footer, button/card components, color & type applied	A styled empty page shell renders on all breakpoints
M2 — Home page	Week 2	Hero, intro, mission/vision, featured, news, stats, CTA	Home fully responsive; preloader + reveal working
M3 — Activities page	Week 3	JSON rendering, search, filters, gallery + lightbox	Adding an activity = editing JSON only
M4 — Achievements page	Week 4	Timeline component, category filtering, certificate lightbox	Timeline responsive incl. mobile single-column
M5 — About page	Week 5	History, objectives, org chart, generations tabs	New generation = one new JSON key
M6 — Polish & launch	Week 6	SEO tags, sitemap, a11y audit fixes, image optimization, cross-browser testing, README, deploy to GitHub Pages	Lighthouse ≥ 90 perf / ≥ 95 a11y; live URL

Process recommendations: use Git from day one (the club is an IT club — the repo is itself a teaching tool); review each milestone on a real phone, not just DevTools; write the content (real text/photos) in parallel with M1-M2, since content delays kill student projects more often than code does.

20. Future Improvements

Frontend-only (no backend needed):

- 1. **Dark mode** — CSS custom properties make this a ~1-day task (`prefers-color-scheme` + toggle stored in `localStorage`).
- 2. **Multi-language toggle** (e.g., English/Khmer) — JSON string dictionaries swapped by JS.
- 3. **Events calendar view** — render `activities.json` into a monthly grid.
- 4. **Search across the whole site** — small client-side index built from the JSON files.
- 5. **Member "generation card" share images** — pre-generated OG images per generation.
- 6. **Progressive Web App** — manifest + service worker for offline viewing and installability.
- 7. **Blog/tutorials section** — Markdown files rendered client-side, showcasing member articles.
- 8. **Contact form via third-party** — Formspree/Netlify Forms give a working form with zero backend code.

If a backend is ever added: 9. Member login area, event registration with capacity tracking, an admin panel replacing JSON editing, and a newsletter.

Suggested additional sections (frontend-only, recommended now):

- **FAQ accordion** on About ("How do I join?", "Do I need experience?") — directly serves the primary persona.
- **Testimonials strip** on Home — quotes from members/alumni build trust fast.
- **Partners/sponsors logo row** — increases credibility with faculty and future sponsors.

Appendix A — Sample Data Schemas

```
// data/activities.json
[
  {
    "id": "web-workshop-2026",
    "title": "Web Development Workshop",
    "category": "Workshop",
    "date": "2026-03-15",
    "description": "Hands-on HTML, CSS, and JavaScript basics for beginners.",
    "image": "assets/images/activities/web-workshop-2026.webp",
    "featured": true
  }
]
```

```
// data/members.json
{
  "2026": [
    { "name": "Member Name", "role": "President", "photo": "assets/images/members/2026/name.webp" }
  ],
  "2025": [
    { "name": "Member Name", "role": "Member", "photo": "" }
  ]
}
```

```
// data/achievements.json
[
  {
    "year": 2025,
    "category": "Competition",
    "title": "1st Place – University Hackathon",
    "description": "Team of 4 built a campus navigation app in 24 hours.",
    "image": "assets/images/achievements/hackathon-2025.webp"
  }
]
```